23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : B10
Team Members :

Roll No	Student Name
AM.SC.U4CSE25167	MYDHILY MS
AM.SC.U4CSE25119	GOUTHAM
AM.SC.U4CSE25159	YADHU
AM.SC.U4CSE25168	MEERA

Project Title : HOSPITAL MANAGEMENT SYSTEM

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	3
2	Introduction	3
3	Requirements (Functional & Non-functional)	3
4	UI Design	4
5	Database Design	5 - 9
6	Database Selection & Justification	9
7	Implementation	10
8	Testing & Validation	11
9	Results & Screenshots	12 - 13
10	Challenges faced & solutions	14
11	Conclusion & future enhancements	14
	References	14

1 Abstract

MediCare is a database application for coordinating patient, clinical and administrative records across hospital branches. It combines a PostgreSQL relational database, an Express REST API and a browser interface. The implementation contains 26 core relations and a separate authentication table, with composite keys, foreign keys, overlap constraints, triggers and transaction-based operations. A reproducible pipeline converts 12 synthetic source registers into normalized records. The verified demonstration includes 300 patients, 90 doctors, 360 appointments and 180 bills. The system supports coordinated record keeping, resource checks and year-based reporting within the scope of a DBMS coursework prototype.

2 Introduction

Separate appointment, ward, pharmacy and billing registers make it difficult to keep hospital information consistent. Repeated patient details introduce update anomalies, while independent schedules can allocate the same resource twice. Our project organizes these activities around shared patient, doctor and branch identifiers.

The strongest features are enforced relationships, branch-specific room and pharmacy identities, preservation of admission history, database-level time-conflict checks, and traceable transformation from raw registers to relational tables. Searchable screens and reports make these database concepts visible during a demonstration.

3 Requirements

Type	Required behavior
Clinical records	Register, search and edit patients and doctors; preserve referenced clinical history when doctors are archived.
Scheduling and care	Book 30-minute consultations within doctor shifts; manage admissions, room release, laboratory results and OT schedules.
Pharmacy and billing	Record prescriptions and pharmacy-specific stock, prices and reorder levels; bill completed appointments and record full invoice settlement.
Administration	Manage branches, departments and pay rules; inspect nurses, staff, equipment, shift logs and salary registers.
Access and reporting	Authenticate users, enforce API role permissions, select report years and export monthly summaries as CSV.
Quality requirements	Use parameterized SQL, validate values, roll back failed multi-step writes, provide readable errors and a consistent responsive interface.

Scope boundaries The project does not provide patient self-service, individual payment receipts, a complete payroll processing wizard or production-level clinical access controls. These are future extensions.

4 UI Design

The interface uses a sidebar for clinical, operational and administrative modules. Users sign in with a seeded role account; navigation and server-side authorization control available operations. Forms use existing record identifiers through selectors, and registers provide search, sorting or pagination where implemented.

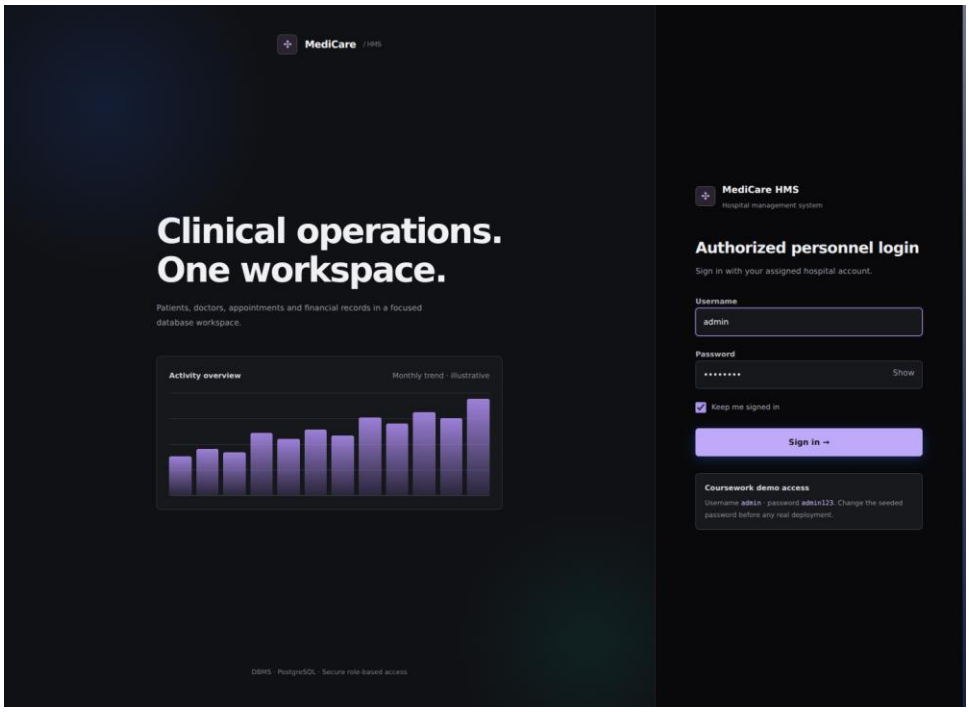


Figure 1 Actual login screen with local coursework credentials and session-persistence option

Screen	Interaction and purpose
Dashboard	Select a year to inspect paid invoice values, appointment counts, patient activity and branch resource indicators.
Patients and doctors	Search records; open entry/edit forms. Doctors can be archived while preserving historical references.
Appointments and admissions	Choose linked records, date and time; receive validation on conflicts. Discharge updates room availability.
Pharmacy and billing	Switch pharmacy inventory; inspect local stock and prices. Create invoices from completed appointments.
Reports and registers	Inspect monthly totals, export CSV and browse laboratory, OT, staffing and payroll records.

Main demonstration flow

Login → patient record → doctor shift → appointment → completion → invoice.
For inpatient care, patient → admission → discharge. Pharmacy inventory and laboratory/OT records are maintained through their own linked modules.

The desktop screenshots in Chapter 9 were captured from the running application using synthetic records. Icons are bundled locally, and the charts are drawn with the application's canvas code.

5 Database Design

5 1 Entity relationships

The following diagrams summarize the implemented foreign-key relationships. An arrow runs from the referenced parent to the child. “1 : N” means each child refers to one parent while a parent may have zero or many children. Full attributes and keys follow in the relational schema.

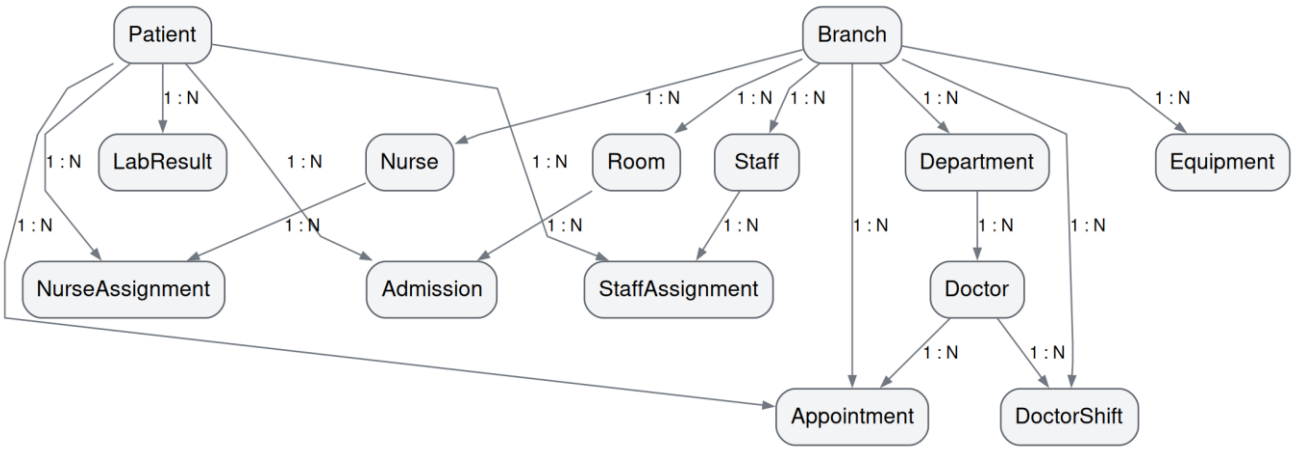


Figure 2 Organization, appointments, inpatient history and care assignments

Room is identified by the pair BranchID and RoomNo. DoctorShift resolves the doctor-to-branch scheduling relationship; NurseAssignment and StaffAssignment record many-to-many care relationships with a date.

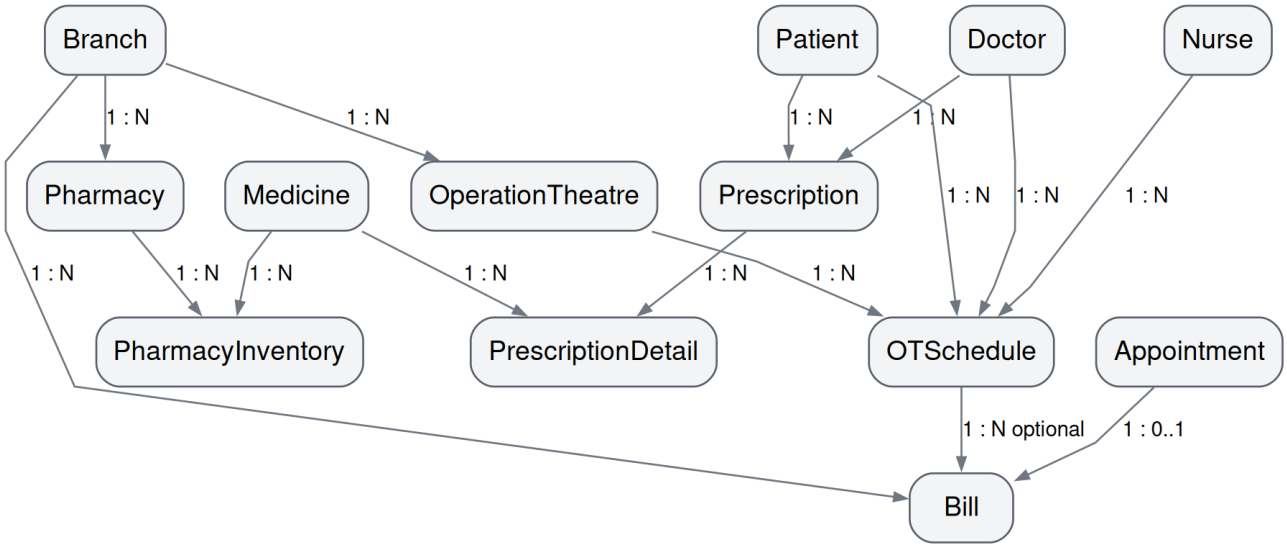


Figure 3 Prescriptions, pharmacy inventory, operating theatres and appointment billing

Each bill references exactly one appointment; the unique appointment key permits at most one bill per appointment. An OT schedule link on a bill is optional. Each prescription has one patient and one doctor, while its medicines are recorded in PrescriptionDetail.

5 2 Payroll relationships and business rules

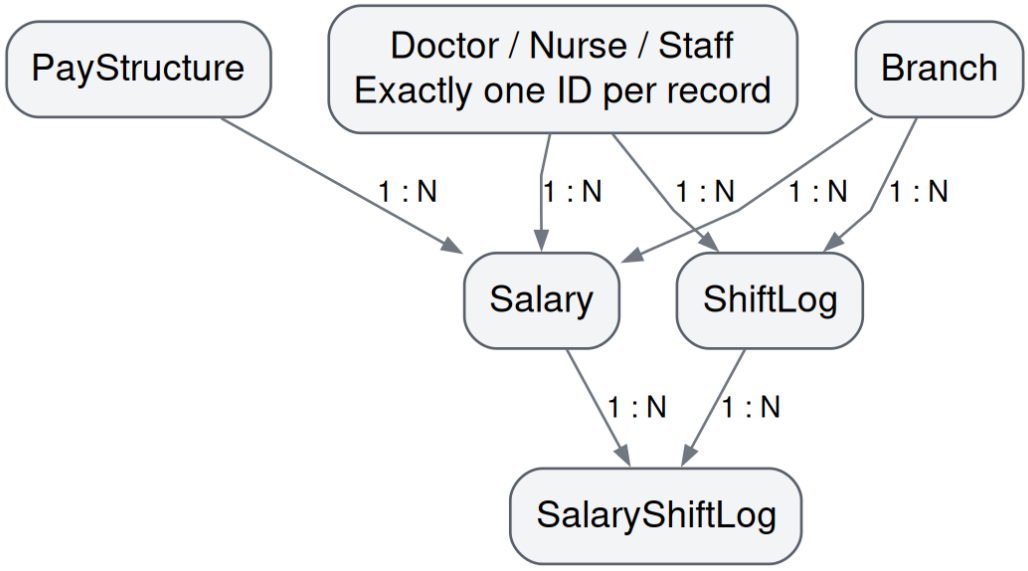


Figure 4 Payroll relationships with exactly one employee type per shift and salary record

UserAccount is an additional authentication relation. It stores username, password hash, role and account status; it is not a clinical employee superclass and has no direct foreign key to Doctor, Nurse or Staff.

Rule	Database or application mechanism
Resource conflicts	GiST exclusion constraints reject overlapping doctor shifts, appointment doctor slots, admissions and OT resource allocations.
Valid consultations	Triggers check active doctor/department/branch status and shift containment. Same-department patient conflicts use department IDs.
Rooms and history	Composite room reference; admission periods cannot overlap for a room or patient. Admission/discharge triggers synchronize current occupancy.
Clinical billing	An invoice requires a completed appointment. Optional operation billing requires a matching completed OT schedule and branch.
Inventory safety	Non-negative stock and prices; atomic conditional decrement refuses insufficient local stock.
Payroll integrity	Exactly one employee ID; active matching pay rule; linked completed shifts match employee, branch and month; salary total equals base plus allowance.

5 3 Conversion from ER model to relations

Strong entities become tables with primary keys. One-to-many relationships place the parent key in the child. Many-to-many relationships become junction tables with composite keys. The weak room identifier becomes (BranchID, RoomNo). Patient address is separated into atomic components; age is calculated from DOB rather than stored.

Exclusion constraints handle time ranges, while triggers express business rules that ordinary foreign keys and row checks cannot capture. The implementation does not yet unify consultation and surgery calendars into a single resource schedule.

5 4 Complete relational schema

Names match the SQL implementation. PK denotes primary key; AK denotes alternate candidate key. An arrow after an attribute identifies its foreign-key target. Admission's branch_id and room_no jointly reference Room.

Relation	Attributes and foreign keys	PK and alternate keys
branch	branch_id, location, status	branch_id
department	department_id, department_name, branch_id → branch, status	department_id
doctor	doctor_id, name, specialization, phone, department_id → department, is_active	doctor_id
doctor_shift	doctor_id → doctor, branch_id → branch, shift_start, shift_end	doctor_id, branch_id, shift_start
patient	patient_id, name, dob, gender, street, city, state, pin_code, phone	patient_id
appointment	appointment_id, patient_id → patient, doctor_id → doctor, branch_id → branch, appointment_date, appointment_time, token_no, status	appointment_id
nurse	nurse_id, name, shift, branch_id → branch	nurse_id
staff	staff_id, name, role, shift, branch_id → branch	staff_id
nurse_assignment	nurse_id → nurse, patient_id → patient, assignment_date	nurse_id, patient_id, assignment_date
staff_assignment	staff_id → staff, patient_id → patient, assignment_date	staff_id, patient_id, assignment_date
room	branch_id → branch, room_no, room_type, critical_care_monitoring, availability_status	branch_id, room_no
admission	admission_id, patient_id → patient, branch_id → room, room_no → room, admit_date_time, discharge_date_time	admission_id
prescription	prescription_id, doctor_id → doctor, patient_id → patient, prescription_date	prescription_id
medicine	medicine_id, name	medicine_id

5 4 Relational schema continued

Names match the SQL implementation. PK denotes primary key; AK denotes alternate candidate key. An arrow after an attribute identifies its foreign-key target. Admission's branch_id and room_no jointly reference Room.

Relation	Attributes and foreign keys	PK and alternate keys
prescription_detail	prescription_id → prescription, medicine_id → medicine, dosage, quantity	prescription_id, medicine_id
pharmacy	pharmacy_id, branch_id → branch	pharmacy_id
pharmacy_inventory	pharmacy_id → pharmacy, medicine_id → medicine, stock_quantity, price, reorder_level	pharmacy_id, medicine_id
lab_result	lab_result_id, patient_id → patient, test_type, order_date, result_date, result_details	lab_result_id
operation_theatre	ot_id, branch_id → branch, status	ot_id
ot_schedule	ot_schedule_id, ot_id → operation_theatre, patient_id → patient, doctor_id → doctor, nurse_id → nurse, procedure, schedule_start, schedule_end, status	ot_schedule_id
equipment	equipment_id, branch_id → branch, name, location, maintenance_date, status	equipment_id
bill	bill_id, appointment_id → appointment, branch_id → branch, amount, bill_date, payment_status, ot_schedule_id → ot_schedule	bill_id; AK appointment_id
shift_log	shift_log_id, branch_id → branch, doctor_id → doctor, nurse_id → nurse, staff_id → staff, shift_date, start_time, end_time, status	shift_log_id
pay_structure	pay_structure_id, employee_type, base_pay, allowance_rate, status	pay_structure_id
salary	salary_id, branch_id → branch, doctor_id → doctor, nurse_id → nurse, staff_id → staff, pay_structure_id → pay_structure, pay_month, base_pay_applied, allowance_applied, salary_amount	salary_id
salary_shift_log	salary_id → salary, shift_log_id → shift_log	salary_id, shift_log_id
user_account	user_id, username, password_hash, role, status	user_id; AK username

Additional candidate keys include Department(branch_id, department_name), Appointment(branch_id, appointment_date, token_no), and each employee's branch/month salary combination through partial unique indexes. Foreign keys reference the parent primary key unless stated otherwise.

5 5 Functional dependencies and normalization

The design separates master descriptions from repeated clinical records. For the core relations, candidate keys determine the stored descriptive attributes. The examples below explain the principal decompositions without repeating every trivial key dependency.

Dependency or decomposition	Reason
PatientID → name, DOB, address fields, phone	Patient identity is stored once and referenced by clinical events. Age remains derived.
DoctorID → DepartmentID; DepartmentID → DepartmentName	Keeping department descriptions in Department removes a transitive dependency from Doctor.
(BranchID, RoomNo) → room type and monitoring	Room numbers may repeat across branches; the full composite key is needed.
(PharmacyID, MedicineID) → stock, price, reorder level	Medicine names belong in Medicine; stock and pricing depend on the complete pharmacy/medicine pair.
(PrescriptionID, MedicineID) → dosage, quantity	Repeating medicine lists become atomic rows with the full prescription/medicine key.
AppointmentID → BillID and bill attributes	AppointmentID is unique and non-null in Bill, making it an alternate key.

1NF Atomic address components and prescription lines remove repeating groups.
2NF Master attributes are separated from composite-key relations.
3NF Department, branch and medicine descriptions remain in their own master tables.
BCNF The listed core determinants are candidate keys or superkeys under the stated assumptions.

Implementation qualification We do not claim blanket BCNF for every physical relation. Salary stores a total determined by applied base pay and allowance; room occupancy is maintained operational state, and billing retains a branch plus an optional OT link. These choices require consistency checks and a qualified normalization claim.

6 Database Selection and Justification

PostgreSQL fits this project because it combines relational keys and joins with transactions, range types, GiST exclusion constraints, PL/pgSQL triggers and reusable views. The btree_gist extension lets resource identifiers participate in time-overlap exclusions. The pgcrypto extension supports hashed demo credentials. Node-postgres provides parameterized queries and client-scoped transactions for the Express API. [1–3]

Using a relational DBMS is appropriate because appointments, prescriptions, admissions and bills depend on explicit relationships and integrity rules. The project's most distinctive database feature is enforcement of time conflicts inside SQL, so invalid direct writes are rejected as well as invalid form submissions.

7 Implementation

The browser sends JSON requests to Express routes on port 5001. Authentication and role middleware run before controllers. Controllers use parameterized SQL against schema hms; database rules protect the stored relationships. The initializer applies three SQL files transactionally, and the loader imports raw staging registers and 26 transformed relations in one transaction.

7 1 Reproducible data pipeline

generate.py → 12 synthetic CSV registers → validate.py → normalize.py → staging and hms tables. The included scale-10 dataset has 1,833 raw rows across 2020–2026. Conflicting values for the same key are rejected; generated identity sequences advance beyond imported IDs.

7 2 Branch workload query and actual output

```
SELECT b.branch_id, b.location, COUNT(a.appointment_id) AS appointments
FROM hms.branch b LEFT JOIN hms.appointment a USING (branch_id)
GROUP BY b.branch_id, b.location ORDER BY b.branch_id;
```

Branch ID	Location	Appointments
101	Kochi Main Hospital	130
102	Ernakulam City Branch	120
103	Thrissur Medical Centre	110

7 3 Low stock query and actual output

```
SELECT pharmacy_id, medicine, stock_quantity, reorder_level
FROM hms.v_pharmacy_low_stock ORDER BY pharmacy_id, medicine_id;
```

Pharmacy	Medicine	Stock	Reorder
701	Paracetamol 500mg	8	20
701	Omeprazole 20mg	8	25
702	Ibuprofen 200mg	8	20
703	Metformin 500mg	8	25

7 4 Safe writes

```
UPDATE hms.pharmacy_inventory
SET stock_quantity = stock_quantity - p_quantity
WHERE pharmacy_id = p_pharmacy_id AND medicine_id = p_medicine_id
  AND stock_quantity >= p_quantity
RETURNING stock_quantity;
```

The dispense function rejects non-positive quantities and raises an error if the update finds insufficient stock. In the integration fixture, stock 10 minus quantity 3 returned 7; a later request for 8 failed and stock remained 7. Appointment token allocation similarly runs on one database client inside a transaction with a branch/date lock.

8 Testing and Validation

All 55 automated tests passed: 23 controller contract tests, 17 generator/validator tests and 15 database/API integration tests. The integration suite creates isolated fixtures and cleans them up; it no longer silently passes when prerequisite data is missing.

Environment PostgreSQL 18.3 through PGlite 0.5.8, Node.js 24.19, the real Express API and Chromium. This verifies SQL behavior and application integration in an embedded PostgreSQL runtime. A native PostgreSQL deployment and concurrent multi-session stress test remain final environment checks. [5]

Test input or action	Expected and observed outcome	Result
No token / wrong password	Protected request / login returns 401.	Pass
Pharmacist creates patient	Role middleware returns 403.	Pass
25 list and report endpoints	All return HTTP 200 with populated fixtures.	Pass
Patient create edit delete	201 on create; updated phone persists; delete succeeds.	Pass
Overlapping doctor booking	First booking succeeds; overlap returns 409.	Pass
Appointment outside shift	Rejected with HTTP 400.	Pass
Delete a booked doctor shift	Database business-rule exception.	Pass
Bill incomplete appointment via SQL	Rejected before invoice creation.	Pass
Bill completed appointment twice	First invoice succeeds; duplicate returns 409.	Pass
Reopen a billed appointment	Completion state change is rejected.	Pass
Settle ₹100 invoice with ₹0 or ₹50	Rejected; full ₹100 settlement succeeds.	Pass
Admit then discharge patient	Room becomes occupied, then available.	Pass
Overlapping / maintenance admission	Conflict or unavailable-room request is rejected.	Pass
Dispense 3 then 8 from stock 10	First leaves 7; second fails without stock loss.	Pass
Overlapping OT schedule	Database exclusion violation.	Pass
Invalid year / valid 2026 report	Invalid input rejected; valid result has 12 months.	Pass

Browser checks Login, 20 administrator screens and six additional role landing pages loaded without uncaught JavaScript errors. Changing the report year and downloading CSV produced a header plus 12 monthly records. Session-only login and a mobile viewport were also exercised. Evidence and exact outputs are included in verification/.

9 Results and Screenshots

The sample database contains 27 application tables: 26 core hospital relations plus UserAccount. The screenshots below were captured from the running application with the loaded synthetic dataset.

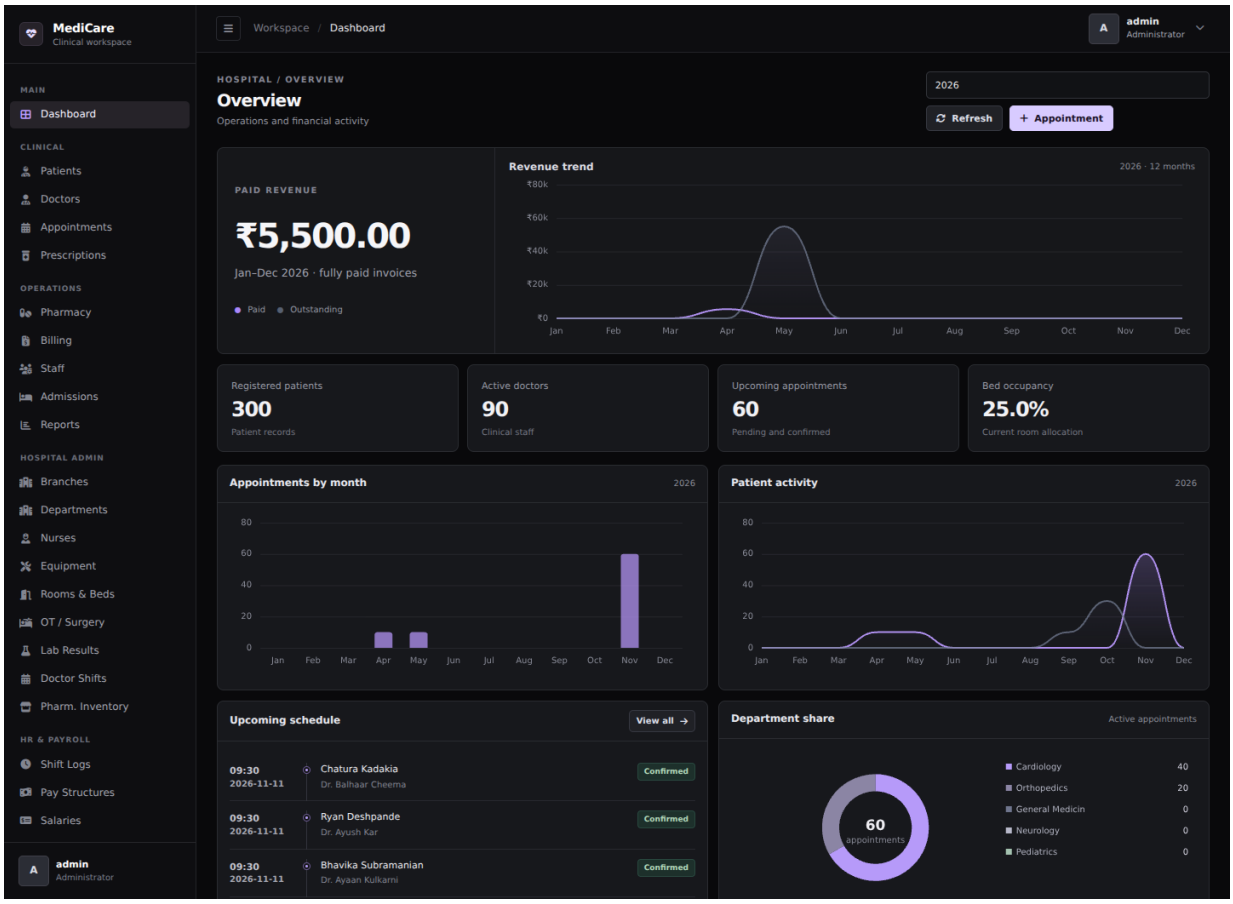


Figure 5 Running dashboard showing 300 patients, 90 doctors, 60 upcoming appointments and 25 percent occupancy at capture time

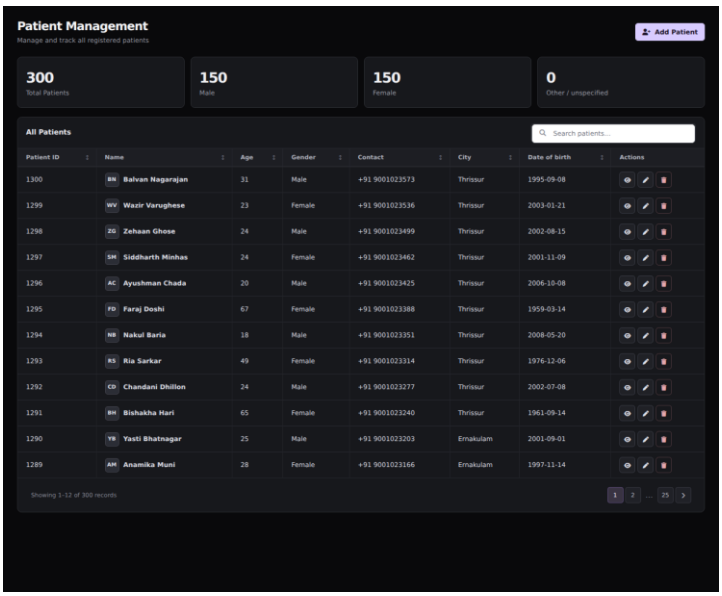


Figure 6 Searchable patient register with pagination and row actions

The patient list, shared identifiers and cross-branch records make the relational design visible. Dashboard totals are database queries; year selection changes the displayed monthly series.

9 1 Pharmacy and reporting results

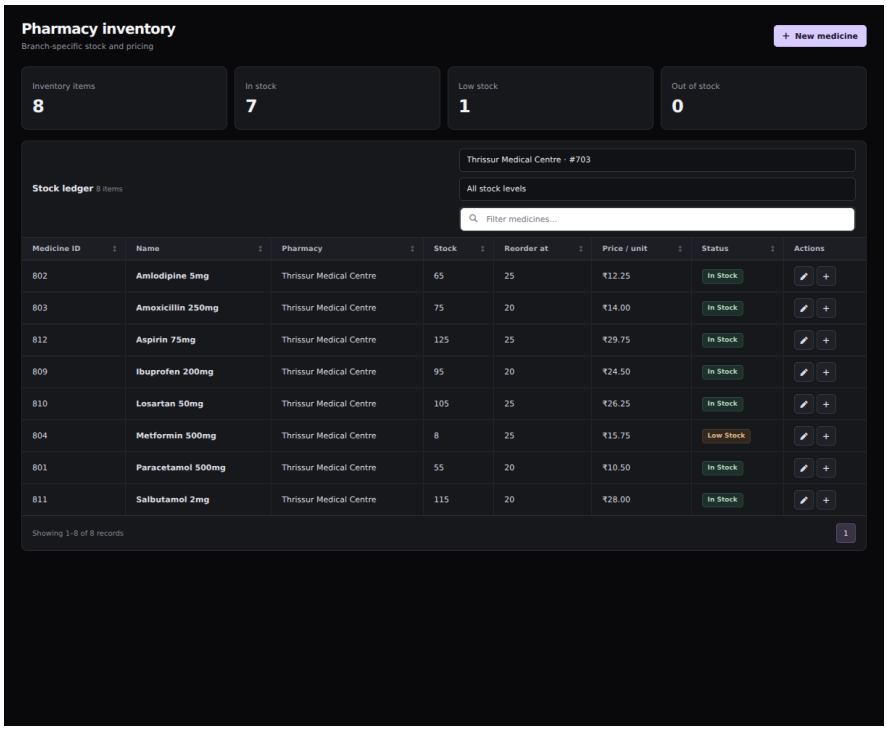


Figure 7 Pharmacy selector exposes branch-specific quantities, prices and low-stock thresholds

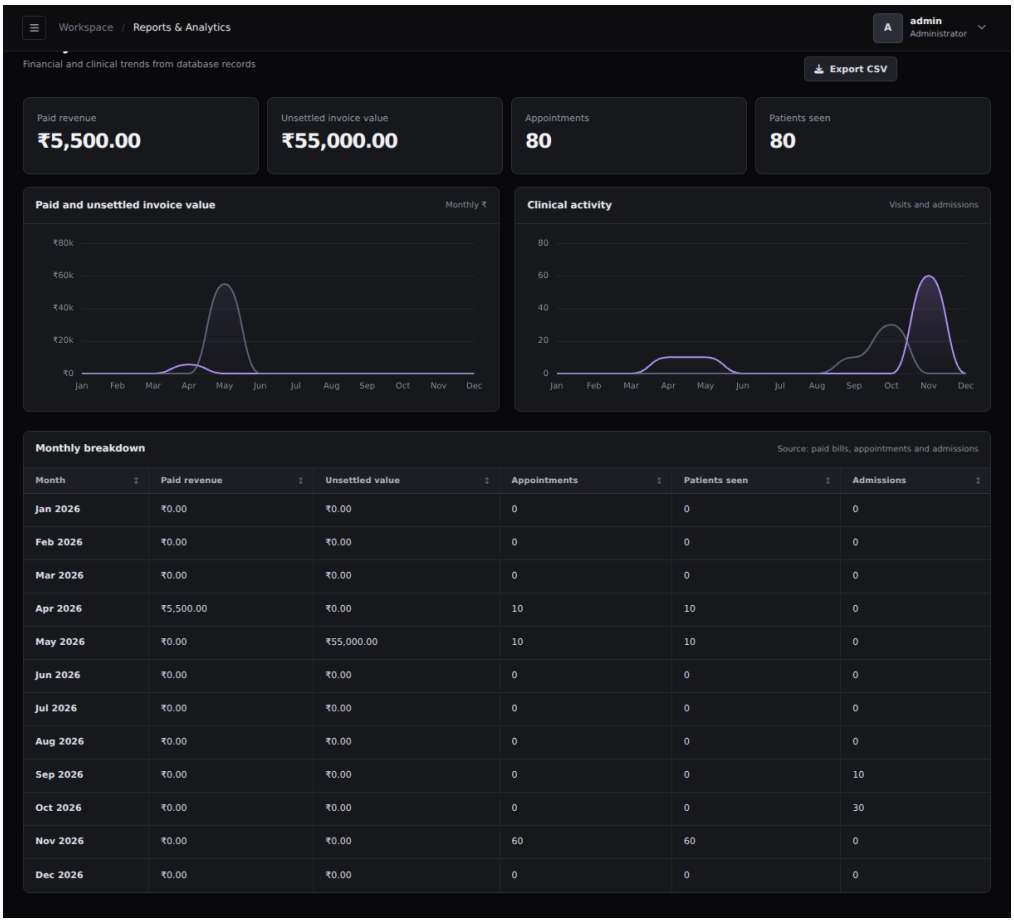


Figure 8 Database-derived monthly report for 2026 with CSV export

For 2026, the captured report shows ₹5,500 in fully paid invoice value and ₹55,000 in unsettled invoice face value. These are grouped by invoice date. Because individual receipts are not stored, “unsettled” is not an exact remaining-payment balance.

10 Challenges Faced and Solutions

Challenge found during review	Solution applied
Broken setup and stale test paths	Added transactional initializer, corrected filenames and documented a repeatable startup.
Room availability diverged from admissions	Added admission/discharge synchronization and unavailable-room guards.
Booking token and shift consistency	Allocated tokens within a locked transaction; blocked changes that invalidate booked shifts.
Billing could accept misleading state	Enforced completed-appointment billing in SQL; constrained the payment action to full settlement.
Dates and offline UI behavior	Preserved calendar values without timezone conversion; bundled icons and repaired login validation.
Weak evidence and overbroad claims	Used real SQL/API/browser checks, deterministic sample data and explicit implementation limits.

11 Conclusion and Future Enhancements

MediCare demonstrates a substantial DBMS application through a connected relational core, meaningful SQL constraints and a usable interface. Its best features are composite resource keys, many-to-many relations, time-overlap exclusions, transactional inventory, normalization evidence, branch-based operations and reproducible reporting. The reviewed build passes the stated automated tests and is suitable for a coursework demonstration after the report cover and local PostgreSQL startup are completed.

Future work should add a receipt ledger with partial-payment balances, a complete payroll workflow, unified consultation/OT conflict checks, employee-specific access, audit history and backup/restore testing. Native multi-user load and concurrency testing should precede any production use.

References

[1] PostgreSQL Global Development Group. PostgreSQL 18 documentation, Constraints. https://www.postgresql.org/docs/18/ddl-constraints.html

[2] PostgreSQL Global Development Group. PostgreSQL 18 documentation, Range Types. https://www.postgresql.org/docs/18/rangetypes.html

[3] node-postgres. Transactions. https://node-postgres.com/features/transactions

[4] Express. Routing guide. https://expressjs.com/en/guide/routing/

[5] Electric. PGlite documentation and extensions. https://pglite.dev/docs/about and https://pglite.dev/extensions/

[6] Supplied project sources and course documents. DBMS-main (3).zip; DBMS-PROJ (1).docx; 04. Project Report.docx.